



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 1

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

*No. de lista o
brigada:* 6

Semestre: 3^{ro}

Fecha de entrega: 28/08/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Motivación	2
1.3. Objetivos	2
2. Marco Teórico	3
3. Desarrollo	4
3.1. Enunciado de salida	4
3.2. Interacción con el LLM y posibles soluciones	4
3.3. Algoritmo mínimo	5
3.4. Diagrama de flujo	5
3.5. Calculadora básica en Java	6
3.6. Algoritmo de la calculadora	6
4. Resultados	7
5. Conclusiones	10

1. Introducción

1.1. Planteamiento del problema

Como continuación de la Práctica 0, en la que se calibró un LLM para que funcionara como tutor, la Práctica 1 constó de dos partes. En la primera parte se utilizó el LLM ya entrenado para analizar la salida de un programa proyectado en clase, a partir de esa foto, solicitar al modelo posibles soluciones o implementaciones capaces de reproducirla. En la segunda parte se diseñó un algoritmo pero ahora sobre un programa propio una calculadora básica con siete operaciones, en la que el comportamiento esperado se definió explícitamente antes de codificarlo.

1.2. Motivación

Usar un LLM previamente contextualizado como asistente para este proceso permite convertir una descripción informal en una propuesta algorítmica estructurada, acelerando la etapa de análisis dentro del ciclo de desarrollo de software.

1.3. Objetivos

- **General:** Aplicar un proceso de análisis, diseño algorítmico y documentación con ayuda de un LLM tanto en la descripción de una salida gráfica dada como al desarrollo de un programa propio, verificando en ambos casos que el algoritmo obtenido reproduzca el comportamiento esperado.
- **Específicos:**
 - Describir de forma clara la salida gráfica observada en el programa proyectado en clase.
 - Formular preguntas al LLM para obtener posibles soluciones o enfoques de implementación de dicha salida.
 - Documentar el algoritmo mínimo necesario para reproducir la salida, apoyándose en un diagrama de flujo.
 - Diseñar y documentar, con el mismo nivel de detalle, el algoritmo de un programa propio (una calculadora básica). sus casos borde.

2. Marco Teórico

El sustento tecnológico y metodológico de la presente práctica se fundamenta en los siguientes principios:

1. **Ingeniería de Prompts (Prompt Engineering):** Técnica de diseño de entradas textuales para condicionar la salida de un LLM hacia un objetivo específico, en este caso la descripción de una imagen y la generación de un algoritmo.
2. **Descripción de salidas gráficas:** Proceso de traducir elementos visuales en lenguaje natural preciso, como paso previo al diseño algorítmico.
3. **Algoritmo:** Secuencia finita y ordenada de pasos que resuelve un problema o reproduce un comportamiento determinado, representable mediante pseudocódigo o diagramas de flujo.
4. **Estructuras de control iterativas y de selección:** el ciclo `do-while` garantiza que un bloque de código se ejecute al menos una vez antes de evaluar la condición de salida, mientras que la estructura `switch` permite seleccionar, entre varias rutas de ejecución, la correspondiente a un valor discreto (en este caso, la operación elegida por el usuario).

3. Desarrollo

3.1. Enunciado de salida

La salida proyectada en clase corresponde a la ventana de un programa de dibujo similar a un editor tipo *Paint*, con una barra de herramientas en la parte superior y un lienzo en blanco debajo. Sobre el lienzo se observan dos elementos:

- Un **rectángulo** ubicado en la esquina superior izquierda, dibujado únicamente con su contorno (sin relleno).
- Un **círculo** de color verde, con relleno sólido, ubicado aproximadamente al centro del lienzo.

3.2. Interacción con el LLM y posibles soluciones

Se planteó al LLM lo siguiente: La salida que se proyecta en el pizarrón muestra una Interfaz Gráfica de Usuario (GUI) muy básica, estructurada como un lienzo de dibujo en 2D. Específicamente, en la pantalla se observa:

- Una barra de herramientas en la parte superior con botones que parecen ser opciones de dibujo (probablemente para seleccionar la figura, deshacer o borrar).
- Un cuadrado (o rectángulo): Dibujado a la izquierda, sin color de relleno (transparente) y con un contorno de línea fina (posiblemente punteada o segmentada).
- Un círculo: Dibujado a la derecha, con un color de relleno verde sólido y sin un contorno de color distinto.

Analiza el siguiente código fuente y describe detalladamente cuál será su salida al ejecutarlo. Como se trata de una interfaz gráfica, especifica paso a paso qué elementos o figuras geométricas se dibujan en la pantalla, sus colores (indicando si es relleno o solo contorno), y su posición relativa en el lienzo.

El LLM propuso, entre otras, las siguientes soluciones:

- Java: Utilizando las bibliotecas Swing (con JFrame y JPanel) y la clase Graphics o Graphics2D para sobrescribir el método paintComponent().
- Python: Utilizando Tkinter (con el widget Canvas) o la biblioteca Turtle para un enfoque más secuencial.
- C / C++: Utilizando bibliotecas como Raylib, SDL o OpenGL (aunque para figuras geométricas básicas en 2D, Raylib o la API gráfica del sistema operativo serían más directas).
- Paso 1: Inicialización del entorno gráfico
Importar las bibliotecas gráficas necesarias.
Crear la ventana principal (Frame/Window) definiendo sus dimensiones (ej. 800×600 píxeles) y título.

- Paso 2: Construcción de la interfaz (GUI) Crear un panel superior para alojar los botones de interacción. Crear el lienzo principal (Canvas/Panel) donde se renderizarán las figuras y añadirlo a la ventana.
- Paso 3: Definición y renderizado del Cuadrado (Contorno) Definir las coordenadas de inicio (x_1, y_1) en el lienzo, además de su ancho y alto. Configurar el "pincel" del sistema gráfico: Establecer el color de trazo (stroke) en gris o negro. Desactivar el color de relleno. Llamar a la función de dibujo de rectángulos (ej. `drawRect(x, y, ancho, alto)`).
- Paso 4: Definición y renderizado del Círculo (Relleno) Definir las coordenadas (x_2, y_2) asegurando que estén a la derecha del cuadrado. Configurar el "pincel" del sistema gráfico: Establecer el color de relleno en verde. Llamar a la función de dibujo de elipses/círculos (ej. `fillOval(x, y, diametro, diametro)`).
- Paso 5: Ciclo de ejecución (Main Loop) Actualizar la pantalla para mostrar los elementos dibujados. Iniciar el ciclo de eventos (event listener) para mantener la ventana abierta hasta que el usuario decida cerrarla.

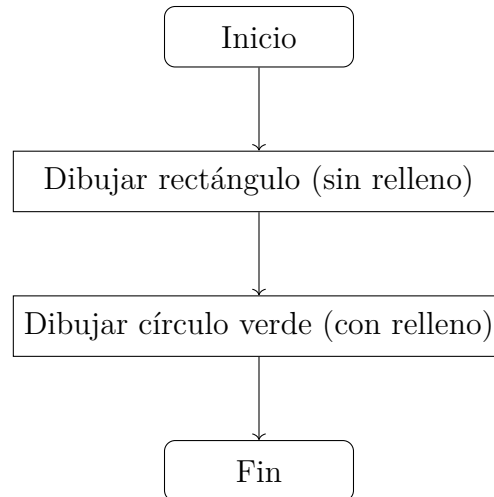
3.3. Algoritmo mínimo

A partir de la solución elegida, se definió el siguiente algoritmo mínimo para reproducir la salida descrita:

1. Iniciar el lienzo (ventana o superficie de dibujo).
2. Definir las coordenadas (x_1, y_1) y dimensiones del rectángulo.
3. Dibujar el rectángulo únicamente con su contorno (sin relleno).
4. Definir el centro (x_2, y_2) y radio del círculo.
5. Dibujar el círculo y rellenarlo con color verde.
6. Finalizar/mostrar el lienzo.

3.4. Diagrama de flujo

El algoritmo anterior se modeló mediante el siguiente diagrama de actividad:



3.5. Calculadora básica en Java

A diferencia de la Parte 1, aquí el comportamiento a algoritmizar no se obtuvo observando un programa ajeno, sino que fue definido directamente como una calculadora que, dados dos números, ejecute la operación que el usuario seleccione de un menú, repitiendo el ciclo hasta que se elija salir del mismo.

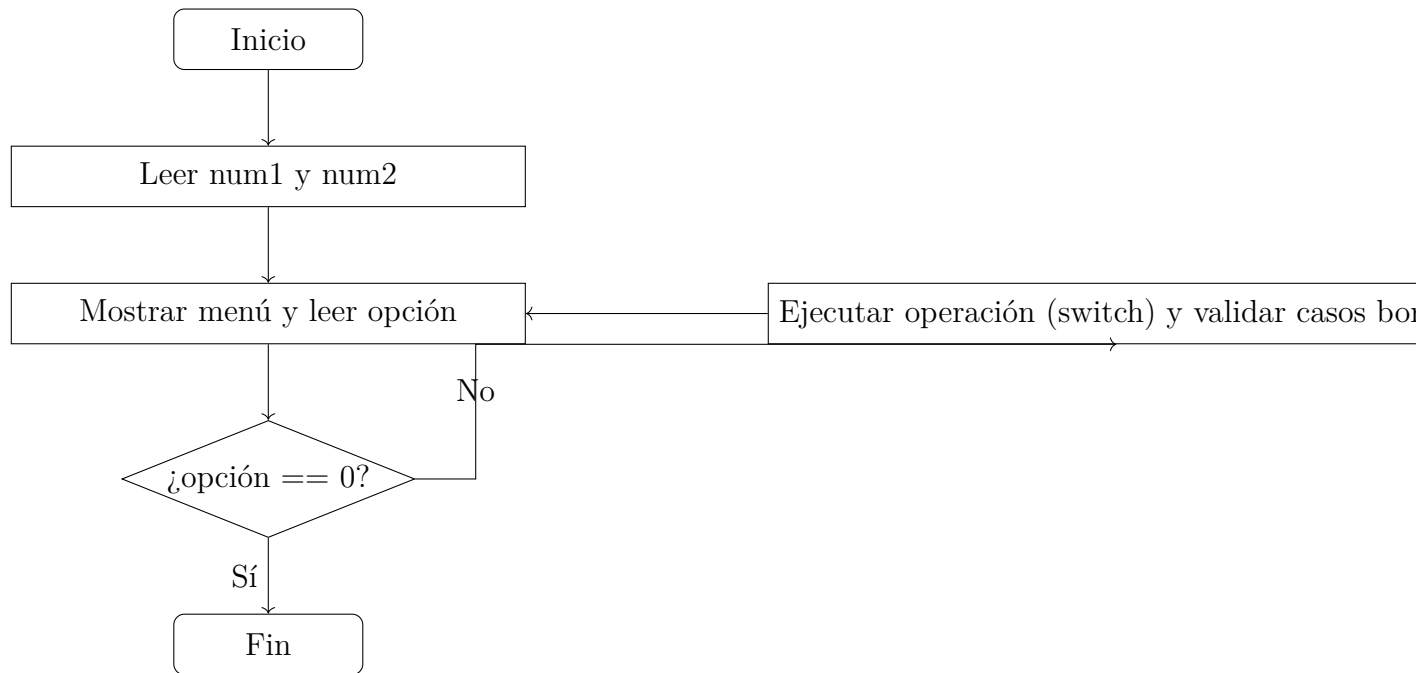
Con base en esto se desarrolló un programa en Java que implementa una calculadora básica con siete operaciones: suma, resta, multiplicación, división, potencia, raíz y módulo. El programa solicita al usuario dos números y despliega un menú en un ciclo `do-while`, dentro del cual una estructura `switch` ejecuta la operación seleccionada hasta que el usuario elige la opción de salida (0).

Para validar el programa se realizaron pruebas con los valores `num1 = 4` y `num2 = 2`, ejecutando cada una de las siete operaciones disponibles:

- **Suma:** entrada (4, 2) → salida 6.0.
- **Resta:** entrada (4, 2) → salida 2.0.
- **Multiplicación:** entrada (4, 2) → salida 8.0.
- **División:** entrada (4, 2) → salida 2.0.
- **Potencia:** entrada (4, 2) → salida 16.0.
- **Raíz:** entrada (4, 2) → salida 2.0 (raíz cuadrada de 4).
- **Módulo:** entrada (4, 2) → salida 0.0.

3.6. Algoritmo de la calculadora

El funcionamiento de la calculadora se resume en el siguiente diagrama de actividad:



4. Resultados

A continuación se muestran las capturas de pantalla de la ejecución del programa, correspondientes a las pruebas descritas en la sección de Desarrollo: la compilación del programa y las siete operaciones de la calculadora.

```
PS C:\Users\o-ale\P00\P00> javac Practica_1.java
PS C:\Users\o-ale\P00\P00> java Practica_1
Bienvenido a la Calculadora Básica
```

Figura 1: Compilación.

```
Ingrese el primer número: 4
Ingrese el segundo número: 2

- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 1

El resultado de la suma de 4.0 mas 2.0 = 6.0
```

Figura 2: Operación suma.


```
- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 2

El resultado de la resta de 4.0 menos 2.0 = 2.0
```

Figura 3: Operación resta.

```
- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 3

El resultado del producto de 4.0 por 2.0 = 8.0
```

Figura 4: Operación multiplicación.

```
- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 4

El resultado de la división de 4.0 entre 2.0 = 2.0
```

Figura 5: Operación división.

```
- Seleccione una operación -  
0. Salir  
1. Suma (num1 + num2)  
2. Resta (num1 - num2)  
3. Multiplicación (num1 * num2)  
4. División (num1 / num2)  
5. Potencia (num1 ^ num2)  
6. Raíz (raíz num2 de num1)  
7. Módulo (num1 % num2)  
Opción: 5  
  
El resultado del producto de 4.0 elevado a 2.0 = 16.0
```

Figura 6: Operación potencia.

```
- Seleccione una operación -  
0. Salir  
1. Suma (num1 + num2)  
2. Resta (num1 - num2)  
3. Multiplicación (num1 * num2)  
4. División (num1 / num2)  
5. Potencia (num1 ^ num2)  
6. Raíz (raíz num2 de num1)  
7. Módulo (num1 % num2)  
Opción: 6  
  
El resultado de la raíz 2.0 de 4.0 = 2.0
```

Figura 7: Operación raíz.

```
- Seleccione una operación -  
0. Salir  
1. Suma (num1 + num2)  
2. Resta (num1 - num2)  
3. Multiplicación (num1 * num2)  
4. División (num1 / num2)  
5. Potencia (num1 ^ num2)  
6. Raíz (raíz num2 de num1)  
7. Módulo (num1 % num2)  
Opción: 7  
  
El resultado del módulo de 4.0 entre 2.0 = 0.0
```

Figura 8: Operación módulo.

```
- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 0

Se salió exitosamente del programa
```

Figura 9: Salir.

5. Conclusiones

A partir de los resultados obtenidos en ambas partes de la práctica, se comprueba que el análisis previo y el modelado de algoritmos son fundamentales para resolver problemas antes de la implementación. En la primera parte, el uso de LLM a través de prompts permitió traducir una salida gráfica abstracta en una secuencia lógica de pasos, demostrando que con unas buenas instrucciones facilita la generación de soluciones.

Por otro lado, los resultados de la calculadora básica demostraron la importancia de aplicar correctamente las estructuras de control. La implementación del ciclo *do-while* fue clave para mantener el programa en ejecución continua, permitiendo al usuario realizar múltiples operaciones sin que la consola se cerrara. Asimismo, la estructura *switch* hizo posible gestionar un menú de opciones de manera limpia y eficiente. Finalmente, los resultados en las pruebas de casos, como la división entre cero o raíces pares de números negativos, reflejan que la validación lógica mediante condicionales es esencial para evitar fallos matemáticos y asegurar que el software responda de forma buena ante entradas inesperadas.

Referencias

- [1] J. White *et al.*, “A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT,” *arXiv preprint arXiv:2302.11382*, 2023.
- [2] H. Schildt, *Java: The Complete Reference*, 12th ed. New York, NY: McGraw-Hill Education, 2021.
- [3] PlantUML Community, “PlantUML Language Reference Guide,” PlantUML Documentation, 2024. [En línea]. Disponible en: <https://plantuml.com/guide>.